

State Estimation Nodes

ekf_localization_node

ekf_localization_node is an implementation of an [extended Kalman filter](#). It uses an omnidirectional motion model to project the state forward in time, and corrects that projected estimate using perceived sensor data.

ukf_localization_node

ukf_localization_node is an implementation of an [unscented Kalman filter](#). It uses a set of carefully selected sigma points to project the state through the same motion model that is used in the EKF, and then uses those projected sigma points to recover the state estimate and covariance. This eliminates the use of Jacobian matrices and makes the filter more stable. However, it is also more computationally taxing than ekf_localization_node.

Parameters

ekf_localization_node and ukf_localization_node share the vast majority of their parameters, as most of the parameters control how data is treated before being fused with the core filters.

The relatively large number of parameters available to the state estimation nodes make launch and configuration files the preferred method for starting any of its nodes. The package contains template launch and configuration files to help get users started.

Parameters Common to *ekf_localization_node* and *ukf_localization_node*

Standard Parameters

`~frequency`

The real-valued frequency, in Hz, at which the filter produces a state estimate.

Note: The filter will not begin computation until it receives at least one message from one of the inputs.

`~sensor_timeout`

The real-valued period, in seconds, after which we consider any sensor to have timed out. In this event, we carry out a predict cycle on the EKF without correcting it. This parameter can be thought of as the inverse of the minimum frequency at which the filter will generate *new* output.

`~two_d_mode`

If your robot is operating in a planar environment and you're comfortable with ignoring the subtle variations in the ground (as reported by an IMU), then set this to true. It will fuse 0 values for all 3D variables (Z, roll, pitch, and their respective velocities and accelerations). This keeps the covariances for those values from exploding while ensuring that your robot's state estimate remains affixed to the X-Y plane.

`~[frame]`

Specific parameters:

- `~map_frame`
- `~odom_frame`
- `~base_link_frame`
- `~base_link_frame_output`

- `~world_frame`

These parameters define the operating “mode” for `robot_localization`. [REP-105](#) specifies three principal coordinate frames: *map*, *odom*, and *base_link*. *base_link* is the coordinate frame that is affixed to the robot. The robot’s position in the *odom* frame will drift over time, but is accurate in the short term and should be continuous. The *map* frame, like the *odom* frame, is a world-fixed coordinate frame, and while it contains the most globally accurate position estimate for your robot, it is subject to discrete jumps, e.g., due to the fusion of GPS data. Here is how to use these parameters:

1. Set the `map_frame`, `odom_frame`, and `base_link_frame` parameters to the appropriate frame names for your system.

Note: If your system does not have a `map_frame`, just remove it, and make sure `world_frame` is set to the value of `odom_frame`.

Note: If you are running multiple EKF instances and would like to “override” the output transform and message to have this frame for its `child_frame_id`, you may set this. The `base_link_frame_output` is optional and will default to the `base_link_frame`. This helps to enable disconnected TF trees when multiple EKF instances are being run. When the final state is computed, we “override” the output transform and message to have this frame for its `child_frame_id`.

2. If you are only fusing continuous position data such as wheel encoder odometry, visual odometry, or IMU data, set `world_frame` to your `odom_frame` value. This is the default behavior for the state estimation nodes in `robot_localization`, and the most common use for it.
3. If you are fusing global absolute position data that is subject to discrete jumps (e.g., GPS or position updates from landmark observations) then:
 - i. Set your `world_frame` to your `map_frame` value
 - ii. **Make sure** something else is generating the *odom*->*base_link* transform. This can even be another instance of a `robot_localization` state estimation node. However, that instance should *not* fuse the global data.

The default values for `map_frame`, `odom_frame`, and `base_link_frame` are *map*, *odom*, and *base_link*, respectively. The `base_link_frame_output` parameter defaults to the value of `base_link_frame`. The `world_frame` parameter defaults to the value of `odom_frame`.

`~transform_time_offset`

Some packages require that your transforms are future-dated by a small time offset. The value of this parameter will be added to the timestamp of *map*->*odom* or *odom*->*base_link* transform being generated by the state estimation nodes in `robot_localization`.

`~transform_timeout`

The `robot_localization` package uses `tf2`’s `lookupTransform` method to request transformations. This parameter specifies how long we would like to wait if a transformation is not available yet. Defaults to 0 if not set. The value 0 means we just get us the latest available (see `tf2` implementation) transform so we are not blocking the filter. Specifying a non-zero *transform_timeout* affects the filter’s timing since it waits for a maximum of the *transform_timeout* for a transform to become available. This directly implies that mostly the specified desired output rate is not met since the filter has to wait for transforms when updating.

`~[sensor]`

For each sensor, users need to define this parameter based on the message type. For example, if we define one source of Imu messages and two sources of Odometry messages, the configuration would look like this:

```
<param name="imu0" value="robot/imu/data"/>
<param name="odom0" value="wheel_encoder/odometry"/>
<param name="odom1" value="visual_odometry/odometry"/>
```

The index for each parameter name is 0-based (e.g., `odom0`, `odom1`, etc.) and must be defined sequentially (e.g., do *not* use `pose0` and `pose2` if you have not defined `pose1`). The values for each parameter are the topic name for that sensor.

~[sensor]_config

Specific parameters:

- ~odomN_config
- ~twistN_config
- ~imuN_config
- ~poseN_config

For each of the sensor messages defined above, users must specify what variables of those messages should be fused into the final state estimate. An example odometry configuration might look like this:

```
<rosparam param="odom0_config">[true, true, false,
                                false, false, true,
                                true, false, false,
                                false, false, true,
                                false, false, false]</rosparam>
```

The order of the boolean values are $X, Y, Z, roll, pitch, yaw, \dot{X}, \dot{Y}, \dot{Z}, \dot{roll}, \dot{pitch}, \dot{yaw}, \ddot{X}, \ddot{Y}, \ddot{Z}$. In this example, we are fusing X and Y position, yaw , $\dot{X}$, and $\dot{yaw}$.

Note: The specification is done in the `frame_id` of the **sensor**, *not* in the `world_frame` or `base_link_frame`. Please see the [configuration tutorial](#) for more information.

~[sensor]_queue_size

Specific parameters:

- ~odomN_queue_size
- ~twistN_queue_size
- ~imuN_queue_size
- ~poseN_queue_size

Users can use these parameters to adjust the callback queue sizes for each sensor. This is useful if your frequency parameter value is much lower than your sensor's frequency, as it allows the filter to incorporate all measurements that arrived in between update cycles.

~[sensor]_differential

Specific parameters:

- ~odomN_differential
- ~imuN_differential
- ~poseN_differential

For each of the sensor messages defined above *that contain pose information*, users can specify whether the pose variables should be integrated differentially. If a given value is set to *true*, then for a measurement at time t from the sensor in question, we first subtract the measurement at time $t - 1$, and convert the resulting value to a velocity. This setting is especially useful if your robot has two sources of absolute pose information, e.g., yaw measurements from odometry and an IMU. In that case, if the variances on the input sources are not configured correctly, these measurements may get out of sync with one another and cause oscillations in the filter, but by integrating one or both of them differentially, we avoid this scenario.

Users should take care when using this parameter for orientation data, as the conversion to velocity means that the covariance for orientation state variables will grow without bound (unless another source of absolute orientation data is being fused). If you simply want all of your pose variables to start at 0, then please use the `_relative` parameter.

Note: If you are fusing GPS information via `navsat_transform_node` or `utm_transform_node`, you should make sure that the `_differential` setting is *false*.

~[sensor]_relative

Specific parameters:

- `~odomN_relative`
- `~imuN_relative`
- `~poseN_relative`

If this parameter is set to `true`, then any measurements from this sensor will be fused relative to the first measurement received from that sensor. This is useful if, for example, you want your state estimate to always start at $(0, 0, 0)$ and with *roll*, *pitch*, and *yaw* values of $(0, 0, 0)$. It is similar to the `_differential` parameter, but instead of removing the measurement at time $t - 1$, we always remove the measurement at time 0, and the measurement is not converted to a velocity.

`~imuN_remove_gravitational_acceleration`

If fusing accelerometer data from IMUs, this parameter determines whether or not acceleration due to gravity is removed from the acceleration measurement before fusing it.

Note: This assumes that the IMU that is providing the acceleration data is also producing an absolute orientation. The orientation data is required to correctly remove gravitational acceleration.

`~gravitational_acceleration`

If `imuN_remove_gravitational_acceleration` is set to `true`, then this parameter determines the acceleration in Z due to gravity that will be removed from the IMU's linear acceleration data. Default is 9.80665 (m/s²).

`~initial_state`

Starts the filter with the specified state. The state is given as a 15-D vector of doubles, in the same order as the sensor configurations. For example, to start your robot at a position of $(5.0, 4.0, 3.0)$, a *yaw* of 1.57, and a linear velocity of $(0.1, 0.2, 0.3)$, you would use:

```
<rosparam param="initial_state">[5.0, 4.0, 3.0,
                                0.0, 0.0, 1.57,
                                0.1, 0.2, 0.3,
                                0.0, 0.0, 0.0,
                                0.0, 0.0, 0.0]</rosparam>
```

`~publish_tf`

If `true`, the state estimation node will publish the transform from the frame specified by the `world_frame` parameter to the frame specified by the `base_link_frame` parameter. Defaults to `true`.

`~publish_acceleration`

If `true`, the state estimation node will publish the linear acceleration state. Defaults to `false`.

`~permit_corrected_publication`

When the state estimation nodes publish the state at time t , but then receive a measurement with a timestamp $< t$, they re-publish the corrected state, with the same time stamp as the previous publication. Setting this parameter to `false` disables that behavior. Defaults to `false`.

`~print_diagnostics`

If `true`, the state estimation node will publish diagnostic messages to the `/diagnostics` topic. This is useful for debugging your configuration and sensor data.

Advanced Parameters

`~use_control`

If *true*, the state estimation node will listen to the `cmd_vel` topic for a `geometry_msgs/Twist` message, and use that to generate an acceleration term. This term is then used in the robot's state prediction. This is especially useful in situations where even small amounts of lag in convergence for a given state variable cause problems in your application (e.g., LIDAR shifting during rotations). Defaults to *false*.

Note: The presence and inclusion of linear acceleration data from an IMU will currently “override” the predicted linear acceleration value.

`~stamped_control`

If *true* and `use_control` is also *true*, looks for a `geometry_msgs/TwistStamped` message instead of a `geometry_msgs/Twist` message.

`~control_timeout`

If `use_control` is set to *true* and no control command is received in this amount of time, given in seconds, the control-based acceleration term ceases to be applied.

`~control_config`

Controls which variables in the `cmd_vel` message are used in state prediction. The order of the values is $\dot{X}$, $\dot{Y}$, $\dot{Z}$, *roll*, *pitch*, *yaw*. Only used if `use_control` is set to *true*.

```
<rosparam param="control_config">[true, false, false,
                                   false, false, true]</rosparam>
```

`~acceleration_limits`

How rapidly your robot can accelerate for each dimension. Matches the parameter order in `control_config`. Only used if `use_control` is set to *true*.

```
<rosparam param="acceleration_limits">[1.3, 0.0, 0.0,
                                         0.0, 0.0, 3.2]</rosparam>
```

`~deceleration_limits`

How rapidly your robot can decelerate for each dimension. Matches the parameter order in `control_config`. Only used if `use_control` is set to *true*.

`~acceleration_gains`

If your robot cannot instantaneously reach its acceleration limit, the permitted change can be controlled with these gains. Only used if `use_control` is set to *true*.

```
<rosparam param="acceleration_limits">[0.8, 0.0, 0.0,
                                         0.0, 0.0, 0.9]</rosparam>
```

`~deceleration_gains`

If your robot cannot instantaneously reach its deceleration limit, the permitted change can be controlled with these gains. Only used if `use_control` is set to *true*.

`~smooth_lagged_data`

If any of your sensors produce data with timestamps that are older than the most recent filter update (more plainly, if you have a source of lagged sensor data), setting this parameter to *true* will enable the filter, upon reception of lagged data, to revert to the last state prior to the lagged measurement, then process all measurements until the current time. This is especially useful for measurements that come from nodes that require heavy CPU usage to generate pose estimates (e.g., laser scan matchers), as they are frequently lagged behind the current time.

`~history_length`

If `smooth_lagged_data` is set to *true*, this parameter specifies the number of seconds for which the filter will retain its state and measurement history. This value should be at least as large as the time delta between your lagged measurements and the current time.

`~[sensor]_nodelay`

Specific parameters:

- `~odomN_nodelay`
- `~twistN_nodelay`
- `~imuN_nodelay`
- `~poseN_nodelay`

If *true*, sets the `tcpNoDelay` [transport hint](#). There is some evidence that Nagle's algorithm interferes with the timely reception of large message types, such as the `nav_msgs/Odometry` message. Setting this to *true* for an input disables Nagle's algorithm for that subscriber. Defaults to *false*.

`~[sensor]_threshold`

Specific parameters:

- `~odomN_pose_rejection_threshold`
- `odomN_twist_rejection_threshold`
- `poseN_rejection_threshold`
- `twistN_rejection_threshold`
- `imuN_pose_rejection_threshold`
- `imuN_angular_velocity_rejection_threshold`
- `imuN_linear_acceleration_rejection_threshold`

If your data is subject to outliers, use these threshold settings, expressed as [Mahalanobis distances](#), to control how far away from the current vehicle state a sensor measurement is permitted to be. Each defaults to `numeric_limits<double>::max()` if unspecified.

`~debug`

Boolean flag that specifies whether or not to run in debug mode. WARNING: setting this to true will generate a massive amount of data. The data is written to the value of the `debug_out_file` parameter. Defaults to *false*.

`~debug_out_file`

If debug is *true*, the file to which debug output is written.

`~process_noise_covariance`

The process noise covariance, commonly denoted Q , is used to model uncertainty in the prediction stage of the filtering algorithms. It can be difficult to tune, and has been exposed as a parameter for easier customization. This parameter can be left alone, but you will achieve superior results by tuning it. In general, the larger the value for Q relative to the variance for a given variable in an input message, the faster the filter will converge to the value in the measurement.

`~dynamic_process_noise_covariance`

If *true*, will dynamically scale the `process_noise_covariance` based on the robot's velocity. This is useful, e.g., when you want your robot's estimate error covariance to stop growing when the robot is stationary. Defaults to *false*.

`~initial_estimate_covariance`

The estimate covariance, commonly denoted P , defines the error in the current state estimate. The parameter allows users to set the initial value for the matrix, which will affect how quickly the filter converges. For example, if users set the value at position $[0, 0]$ to a very small value, e.g., $1e-12$, and then attempt to fuse measurements of X position with a high variance value for X , then the filter will be very slow to “trust” those measurements, and the time required for convergence will increase. Again, users should take care with this parameter. When only fusing velocity data (e.g., no absolute pose information), users will likely *not* want to set the initial covariance values for the absolute pose variables to large numbers. This is because those errors are going to grow without bound (owing to the lack of absolute pose measurements to reduce the error), and starting them with large values will not benefit the state estimate.

`~reset_on_time_jump`

If set to *true* and `ros::Time::isSimTime()` is *true*, the filter will reset to its uninitialized state when a jump back in time is detected on a topic. This is useful when working with bag data, in that the bag can be restarted without restarting the node.

`~predict_to_current_time`

If set to *true*, the filter predicts and corrects up to the time of the latest measurement (by default) but will now also predict up to the current time step.

`~disabled_at_startup`

If set to *true* will not run the filter on start.

Node-specific Parameters

The standard and advanced parameters are common to all state estimation nodes in `robot_localization`. This section details parameters that are unique to their respective state estimation nodes.

`ukf_localization_node`

The parameters for `ukf_localization_node` follow the nomenclature of the [original paper](#) and [wiki article](#).

- **~alpha** - Controls the spread of sigma points. Unless you are familiar with unscented Kalman filters, it's probably best for this setting to remain at its default value (0.001).
- **~kappa** - Also control the spread of sigma points. Unless you are familiar with unscented Kalman filters, it's probably best for this setting to remain at its default value (0).
- **~beta** - Relates to the distribution of the state vector. The default value of 2 implies that the distribution is Gaussian. Like the other parameters, this should remain unchanged unless the user is familiar with unscented Kalman filters.

Published Topics

- `odometry/filtered` ([nav_msgs/Odometry](#))
- `accel/filtered` ([geometry_msgs/AccelWithCovarianceStamped](#)) (if enabled)

Published Transforms

- If the user's `world_frame` parameter is set to the value of `odom_frame`, a transform is published from the frame given by the `odom_frame` parameter to the frame given by the `base_link_frame` parameter.
- If the user's `world_frame` parameter is set to the value of `map_frame`, a transform is published from the frame given by the `map_frame` parameter to the frame given by the `odom_frame` parameter.

Note: This mode assumes that another node is broadcasting the transform from the frame given by the `odom_frame` parameter to the frame given by the `base_link_frame` parameter. This can be another instance

of a `robot_localization` state estimation node.

Services

- `set_pose` - By issuing a [geometry_msgs/PoseWithCovarianceStamped](#) message to the `set_pose` topic, users can manually set the state of the filter. This is useful for resetting the filter during testing, and allows for interaction with `rviz`. Alternatively, the state estimation nodes advertise a `SetPose` service, whose type is [robot_localization/SetPose](#).